

Document Version: 1.0 (a newer version number means an update on the project)

Assignment Date: 4/2/2026, Noon

Due Date - No Penalty: 4/16/2026, 11:59pm

Due Date - 10% Penalty: 4/23/2026, 11:59pm

No submission is accepted after 4/23/2026, 11:59pm due to the end of the semester.

Group Size: 1 to 3 Students; single submission for each group

Objectives:

- Continue writing non-trivial C programs
- Continue reading man pages
- Exercise with low-level file IO system calls
- Practice formatting and mounting file systems
- Exercise with file IO related Linux commands
- Perform and report file IO experiments

1 (50 pts) Project Description

In this project you will write your own file io testing tool called `myio`, which will allow you to experiment with low-level file IO system calls in Linux including but not limited to `open(2)`, `read(2)`, `write(2)`, and `close(2)`. You are not allowed to use higher-level file IO API calls from the 3rd section of the man pages, such as `fopen(3)`, `fclose(3)`, `fread(3)`, `fwrite(3)`, `fputs(3)`, `fgets(3)`, `fprintf(3)`, `fscanf(3)`, etc., unless explicitly instructed to use a certain library function in the project description below.

In addition to submitting the source file of your `myio` tool, you will also prepare and submit a report including your experimental results supported with screenshots as well as your descriptions and interpretations of the experiments that you performed as instructed in the rest of the document. **This report will only be accepted in the PDF form.**

Your `myio` tool can be executed in two different modes: (i) to create a file, and (ii) to perform random IO on a file, as described below.

1.1 (25 pts) File Creation Mode

In order to create a file with `file_size` blocks, your program will be invoked as follows:

```
./myio 1 filename file_size
```

The first parameter (1) indicates that the file creation mode will be used, the second parameter includes the name of the file to be created, and the third parameter indicates the size of the file to be created in blocks (not bytes). Block size will be assumed to be 4094 bytes (`#define` it at the beginning of your program). Therefore, the size of the created file will be `file_size` x 4096 bytes (you can check the size at the end with the `ls -l` command). Your `myio` tool in file creation mode will not print anything to the screen.

You will use the `write(2)` system call and the content of the file will be all 0s. You can use `memset(3)` for this purpose (check `man memset`). Since it will only include binary 0s (the NUL character in ASCII), the file will not be in a human readable form but it will allow us to use it as a virtual disk as well as to perform random IO experiments on it. If the file already exists then truncate it to length 0 before writing anything. Also, make sure that the owner as well as the group members have the read/write permissions while others only have the read permission. Check `man 2 open` in detail for more information on these. Finally, make sure to call `fsync` at the end (check `man fsync`) to synchronize your file's in-core state with the storage device. However, also note that making too many `fsync` calls will slow down your program, so read and learn more about `fsync` from the man pages. **Reading the man pages is one of the learning objectives of this project and it will greatly help you throughout the project!**

1.2 (25 pts) Random IO Mode

In order to perform random IO on a given file, your program will be invoked as follows:

```
./myio 2 filename io_size samples
```

The first parameter (2) indicates the random IO mode, the second parameter includes the name of an existing file to perform random IO on, the third parameter indicates the IO size to be performed in bytes, and the last parameter indicates the number of random IO samples your program will collect. You will calculate the amount of time that it takes to perform each random IO **in microseconds** using `clock_gettime(3)`, which we will call the IO latency, and print the median of all IO latency samples that you collected at the end with exactly two decimal points followed by a newline ("%0.2f\n"). This will be the only information that your program will print to the screen. Please note that it is more suitable to use the `CLOCK_MONOTONIC` clock of `clock_gettime(3)` for measuring IO latency (more info in `man clock_gettime`).

In random IO, the goal is reading random `io_size` chunks from the given file, where each read starts at a random byte location. You can use a random number generator in C (such as `random(3)` and `srandom(3)`) to determine the starting byte location for each random IO. However, make sure that you do not read outside the valid bytes of a file (you should check the size of the file in bytes using `stat(2)` to make sure of that), each read is exactly `io_size` bytes, and the beginning byte location of random IO is always a multiple of block size, which is defined to be 4096 in your program. This way, your random IO will be aligned with file system blocks. Once you determine the beginning byte position of random IO, your program will first seek to that position using the `lseek(2)` system call and then will perform a read operation using the `read(2)` system call. Alternatively, the same task can also be performed in a single system call using `pread(2)`, which combines `lseek(2)` and `read(2)`. IO latency measurement will include the time spent in `lseek+read`, so your measurement will start right before calling `lseek` and will end right after calling `read`, or if you are using `pread(2)` instead, your measurement will start right before calling `pread(2)` and will end right after calling `pread(2)`. You are allowed to use `qsort(3)` for the median calculation of your samples.

1.3 (50 pts) File IO Experiments

In the rest of the project, you will not do much coding but you will practice with various Linux commands, formatting and mounting file systems as well as performing file IO experiments with the programs that you wrote previously. To get credit, your experiments need to be supported by terminal screenshots and shared in your PDF report together with a supporting text as described below.

1.3.1 (5 pts) Experiment 1 (File Creation)

First, run your `myio` program in file creation mode to generate files with multiple sizes, including some large ones in the order of GBs. Next, use the `dd` command of Linux to generate another file with an identical content and compare your file content with the content of the file that was generated by `dd` using the `diff` command. Check the man pages of these commands as well the provided BlackBox testing script to learn more about how to execute them. Finally, measure the execution time of your program compared with `dd`'s execution time using the `time` command and report your execution time results. Comment on these results if your program is considerably slower than `dd`, especially for larger files in GBs of size.

1.3.2 (5 pts) Experiment 2 (Formatting and Mounting FAT32)

Run your `myio` program in file creation mode to generate a large file (in 100s of MBs or GBs) that will be used as a virtual disk. Next, use the `losetup` command to associate a loop device (such as `/dev/loopX` where `X` is a number) with your virtual disk file. This way, your virtual disk file will look like a block storage device.

Check the man pages of `loop(4)` and `losetup(8)` to learn more about loop devices and how to use the `losetup` command. Check also the provided BlackBox testing script to see how we use the `losetup` command in our tests. In the BlackBox testing script, only consider the command provided inside the parameters (if parameters are used), such as the following `losetup` command usage:

```
sudo losetup -find -show -nooverlap -direct-io=on fat32disk.img
```

and ignore the new variable creation part with the return value of the command. Just record the loop device associated with your virtual disk somewhere so that you can use it when you need to detach the association later. Document the creation of your loop device in your report using a screenshot.

Each device in Linux has a device file associated with it starting with the `/dev` prefix. After running the `losetup` command as shown above, your virtual disk file is now treated as a block oriented storage device that can be formatted with a file system and mounted. Now, you will format your virtual disk with FAT32 file system using the `mkfs.fat` command as shown in the BlackBox testing script. Make sure to check the man page of the `mkfs.fat(8)` to learn more about the parameters we used, and explain the meaning of each parameter in your report, together with your terminal screenshot showing the completion of formatting.

Once you formatted your virtual disk with the FAT32 file system now you have a FAT32 volume that can be mounted and used. First, create a mount point, that means create an empty directory with the `mkdir` command. The name of the empty directory can be `myfat32fs` as in the provided BlackBox testing script. Then mount your volume on this mount point as also shown in the script. Instead of the `$loopfat32` bash variable that we use in the script, you will use the name of the loop device associated with your virtual disk. Your initial `losetup` command should have returned the name of the associated loop device, but if you want to double check, then you can run the `losetup` command with the `-a` option. Check the man page of `losetup` again for more information. You can now change into `myfat32fs` directory. That means you are changing into the root directory of the FAT32 volume you mounted. There you can create a file with your `myio` tool's file creation mode as shown in the line 26 of the provided BlackBox testing script. Provide a screenshot showing the completion of the mount command as well as the file creation.

1.3.3 (5 pts) Experiment 3 (Formatting and Mounting ext4)

Repeat everything that you did in Experiment 2, but instead of formatting your virtual disk with the FAT32 file system, now you will format it with the ext4 file system. The second half of the provided BlackBox testing script shows the set of commands that we used for this task, which are mostly similar to the ones that you run in Experiment 2 with some changes specific to the ext4 file system. Analyze and report these changes in your PDF, and also apply them to complete this step. Similar to Experiment 2, Experiment 3 also ends with running your `myio` program in file creation mode as in the line 70 of the provided BlackBox testing script. Support your work with screenshots as in Experiment 2.

1.3.4 (5 pts) Experiment 4 (Formatting and Mounting xfs)

Repeat everything that you did in Experiment 2, but instead of formatting your virtual disk with the FAT32 file system, now you will format it with the xfs file system. There is no BlackBox testing script showing you the specific commands for xfs, so it is your job to come up with the correct commands and tools, some of which might need to be installed first. Share the specific commands/tools that you used in your report. Similar to Experiment 2, Experiment 4 also ends with running your `myio` program in file creation mode. Support your work with screenshots as in Experiment 2.

1.3.5 (5 pts) Experiment 5 (Formatting and Mounting f2fs)

Repeat everything that you did in Experiment 2, but instead of formatting your virtual disk with the FAT32 file system, now you will format it with the f2fs file system. There is no BlackBox testing script showing you the specific commands for f2fs, so it is your job to come up with the correct commands and tools, some of which might need to be installed first. Share the specific commands/tools that you used in your report. Similar to Experiment 2, Experiment 5 also ends with running your `myio` program in file creation mode. Support your work with screenshots as in Experiment 2.

1.3.6 (20 pts) Experiment 6 (Random IO Experiments)

Perform some random IO on the files that you created in the last steps of Experiments 2-5 using the random IO mode of your `myio` program. Again, check the BlackBox testing script to see example runs for some file systems. Before each run of your `myio` program, make sure to clear the cache using the following command:

```
sudo sh -c 'sync && echo 3 > /proc/sys/vm/drop_caches'
```

Also, make sure that your total requested IO size is not very large compared to the file size you experiment with. When this happens, then many IO requests will be answered by the RAM instead of the storage device providing you unrealistically fast IO times. Therefore, always make sure that your sample size multiplied with the io size (`io_size x samples`) is less than 5% of your file size.

As part of this experiment, try experimenting with the `io_sizes` of 4096, 8192, 16384, 32768, and 65536, and plot your result with your favorite plotting tool where the x-axis show the io-size and the y-axis shows the received median io-time. You will have four plots, one for each file system. Provide these plots in your report together with supporting screenshots showing your runs. Provide some insights on these results and let us know if they make sense to you. To do that, check the vendor and model name of your storage device (you can run `udisksctl status` to learn that), and try to find its spec sheet from the manufacturer website, which may include expected latency values for 4K random reads. Alternatively, you can also experiment with the `fio` tool used in the BlackBox testing script to run similar experiments on your files with a different io tool to see if your results match. You need to install `fio` to be able to use it (`sudo apt-get install fio`). Installing `fio` is also necessary for running the provided BlackBox testing script.

For you to have an idea on the latency numbers, here is the output that we received from the provided BlackBox testing script by running it on a laptop including a samsung nvme ssd:

```
./test.sh
```

This BlackBox testing script requires around 4GB of free disk space and may take around 15 seconds (for SSDs) or longer (for HDDs) depending on your storage device type!

```
Test 1 - Success-----fat32disk.img-----Success
myio-fat32-4K: 51.05
fio-fat32-4K: 50.43
Test 2 - Success-----fat32-latency-----Success
Test 3 - Success-----ext4disk.img-----Success
myio-ext4-8K: 67.71
fio-ext4-8K: 66.05
Test 4 - Success-----ext4-latency-----Success
```

1.3.7 (5 pts) Experiment 7 (Cleaning)

In this experiment, you will be doing some cleaning. Once you are done with all experimentation, you can unmount the previously mounted volume using the `umount` command, and delete the association between the loop device file and the virtual disk file using the `-d` option of the `losetup` command as shown at the end of the BlackBox testing script. In this script, we pass a bash variable created with the return value of the initial `losetup` run as the parameter right after `-d`, but you can just pass the name of the loop device that the initial `losetup` run returned, or you found out by running `losetup` with `-a` option. Finally, remove the root directories of your file systems and everything inside them, as well as your virtual disk files using the `rm` common as shown in the script.

2 Development

You will develop your program in a Unix environment using the C programming language. You can develop your program using a text editor (emacs, vi, gedit etc.) or an Integrated Development Environment available in Linux. `gcc` will be used as the compiler. You will be provided a Makefile and your program should compile without any errors/warnings using this Makefile.

Black-box testing will be applied and your program's output will be compared to the correct output. A simple black-box testing script is provided to you for your own test; make sure that your program produces success messages in the provided test. A more complicated test (possibly more than one test) might be applied to grade your program. Submissions not following the specified rules here will be penalized. The provided black-box test will require you to enter your sudo password to be able to clear the cache before each execution of an io benchmark. We will also consider your specific io times as in Tests 3 and 4 compared to fio values, to check if your tool is generating expected performance results.

2.1 Checking Memory Leaks

You will need to make dynamic memory allocation. If you do not deallocate the memory that you allocated previously using `free()`, it means that your program has memory leaks. To receive full credit, your program should be memory-leak free. You can use `valgrind` to check the memory-leaks in your program. `valgrind` will output:

"All heap blocks were freed - no leaks are possible"

if your program is memory-leak free.

3 Submission

Submission will be done through Blackboard strictly following the instructions below. Your work will be penalized 5 points out of 100 if the submission instructions are not followed. In addition, memory leaks will be penalized with a total of 5 points without depending on the amount of the leak. Similarly, compilation warnings will also be penalized with a total of 5 points without depending on the type or the number of warnings. You can check the compilation warnings using the `-Wall` flag of `gcc`.

3.1 What to Submit

1. `myio.c`: Source code of your io testing program.
2. `Makefile`: Makefile used to compile your program. If different than the provided one, then you should inform the TA about your changes.

3. `report.pdf` including the following information:
 - Names and IDs of the group members.
 - Did you fully implement the project as described? If not, what parts are not implemented at all or not implemented as following the specified description?
 - For you Random IO experiments, did you use the `lseek+read` approach, or the `pread` approach, and why?
 - Your File IO Experimental Results as described in Section 1.3. Remember that the quality of this report determines 50% of your grade, so make sure you provide sufficient detail. Include clear descriptions and interpretations of the experiments you performed as instructed, supported by screenshots that clearly show you ran them on your own machine. The screenshots should include the terminal displaying your machine name and username. We may ask you to perform a demo in that machine, if needed.

3.2 How to Submit

1. Create a directory and name it as the project number combined with your UofL ID number. For example, if a student's ID is 1234567, then the name of the directory will be ProjectX-1234567 for project X. If it is a group project, then use only one of the group member's ID and we will get the other student's ID from the README file.
2. Put all the files to be submitted (only the ones asked in the *What to Submit* section above) inside this directory.
3. Zip the directory. As a result, you will have the file ProjectX-1234567.zip.
4. Upload the file ProjectX-1234567.zip to Blackboard using the "Attach File" option. You do not need to write anything in the "Submission" and "Comments" sections.
5. **Your project will be graded only once!** If you make multiple attempts of submission before the deadline, then only your latest attempt will be graded.

4 Grading

Grading of your program will be done by an automated process. Your programs will also be passed through a copy-checker program which will examine the source codes if they are original or copied. We will also examine your source file(s) manually to check if you followed the specified implementation rules, if any.

5 Collaboration and Code Sharing Policy

This course assumes that assignments submitted by students will be generated by the students themselves working individually. This policy specifies that the following constitute violations of academic honesty: a student has another person or entity complete any portion of a graded assignment for them, which includes purchasing work from a company, hiring a person or company to complete an assignment or exam, **and/or fully or partially copying code from other resources (such as the Internet) or generating code using AI tools (such as ChatGPT, Gemini, GitHub Copilot, DeepSeek R1, Claude Sonnet, Cursor, etc.).** Except for parts provided by us or the book, if any, all submitted code must be written by *you* and must not be shared. You are not allowed to review your classmates' or friends' code, and they are not allowed to see your code. You may only have high-level verbal discussions about the project with other students. In addition, you are not allowed to publish your code partially (in forums or other sites for asking questions) or fully (in public version control services such as GitHub, etc.). **You cannot post your code on your public GitHub account unless you make it private!** You are also not allowed to share your code with future students of this course. **You may NOT use Chegg.com, CourseHero.com, MyAssignmentHelp.com, or any other sites to post your**

assignment/code or receive any help. Failure to comply with these collaboration and code-sharing rules will put you and those with whom you share your code at risk of plagiarism for this or future semesters. Cheating or copying assignments (including from Internet resources and generative AI tools) will be reported to the dean's office for plagiarism, and a grade of F will be recorded for the course. **If you are unsure whether a certain practice would put you at risk of plagiarism, check with the instructor in advance. A simple MS Teams chat with the instructor will suffice!**

6 Changes

- No changes.